Square of side N

Easy

Problem Statement: Write a program to print a solid square pattern of size N using the '*' character.

Objective: Understand nested loops and basic 2D spatial printing.

Input Format	A single integer N.
Output Format	N lines, each containing N asterisks separated by a space.
Constraints	`0 <= N <= 20`

Example Input	`3`
Example Output	`* * *` `* * *` `* * *`
Explanation	For N=3, we need 3 rows and 3 columns of stars.

Edge Cases to Consider: N=1 (single star), N=0 (empty output).

Additional Test Cases:

```
1) N=1 -> *
2) N=2 -> * *
          * *
```

Hint: Use an outer loop for rows and an inner loop for columns.

> **Intuition & Analogy**
>
> *Analogy: Think of building a brick wall. The outer loop is adding a new layer (row) to the wall, and the inner loop is placing individual bricks (stars) side by side in that layer.*

Right Angled Triangle

Easy

Problem Statement: Write a program to print a right-angled triangle of height N using '*'.

Objective: Master loop dependency where the inner loop's limit depends on the outer loop variable.

Input Format	A single integer N.
Output Format	N lines, where the i-th line contains i asterisks.
Constraints	`0 <= N <= 30`

Example Input	`4`

Example Output	`*`
	`* *`
	`* * *`
	`* * * *`

Explanation	Row 1 has 1 star, Row 2 has 2 stars, etc.

Edge Cases to Consider: N=0 should print nothing.

Additional Test Cases:

```
1) N=2 -> *
          * *
```

Hint: Inner loop should run from 1 to the current row number.

> **Intuition & Analogy**
>
> *Analogy: Think of a staircase. Each step you take downwards (outer loop), the staircase gets wider by exactly one block (inner loop).*

Real-World: Stadium Seating Pattern

Medium

Problem Statement: A stadium is organizing its seating such that the first row has 1 seat, the second has 3 seats, the third has 5 seats (a pyramid). Write a program to print the seating arrangement using 'S' for seats.

Objective: Apply pattern logic to a real-world alignment problem (Pyramid pattern).

Input Format	Integer N (number of rows).
Output Format	N lines of 'S' centered like a pyramid.
Constraints	`1 <= N <= 15`

Example Input	`3`

Example Output	`  S`
	`SSS`
	`SSSSS`

Explanation	Row 1 needs 2 spaces, 1 S. Row 2 needs 1 space, 3 S. Row 3 needs 0 spaces, 5 S.

Edge Cases to Consider: Handling spacing correctly for double-digit N.

Additional Test Cases:

```
1) N=2 ->
 S
SSS
```

Hint: Calculate the number of spaces needed before the 'S' characters for each row.

> **Intuition & Analogy**
>
> *Analogy: Think of framing a triangular roof. You need to pad the empty air (spaces) on the left side to keep the roof centered before you start laying the tiles ('S').*

Celsius to Fahrenheit

Easy

Problem Statement: Write a function `celsius_to_fahrenheit(c)` that converts Celsius to Fahrenheit.

Objective: Learn to implement mathematical formulas inside reusable functions.

Input Format	A float representing temperature in Celsius.
Output Format	A float representing temperature in Fahrenheit, rounded to 2 decimal places.
Constraints	`-273.15 <= C <= 1000`

Example Input	`0`
Example Output	`32.00`
Explanation	Formula: F = (C * 9/5) + 32

Edge Cases to Consider: Absolute zero (-273.15).

Additional Test Cases:

```
1) C=100 -> 212.00
2) C=-40 -> -40.00
```

Hint: Use the built-in round() function.

> **Intuition & Analogy**
>
> *Analogy: A function is like a specialized oven. You put raw ingredients (arguments like Celsius) into the oven, it applies a specific recipe (the formula), and hands you a finished cake (the return value).*

Distance Covered by a Vehicle

Easy

Problem Statement: Write a function `calculate_distance(speed, time)` that returns the distance.

Objective: Practice using multiple parameters in functions.

Input Format	Two numbers (speed in km/h, time in hours).
Output Format	A float representing distance in km.
Constraints	`speed >= 0, time >= 0`

Example Input	`60, 2.5`
Example Output	`150.0`
Explanation	Distance = speed * time = 60 * 2.5 = 150

Edge Cases to Consider: Time or speed is 0.

Additional Test Cases:

```
1) speed=0, time=10 -> 0.0
```

Hint: Simply multiply the two parameters.

> **Intuition & Analogy**
>
> *Analogy: If speed is your hourly wage and time is hours worked, the function calculates your total paycheck (distance).*

Real-World: Delivery Route Estimator

Medium

Problem Statement: Write a function `route_cost(distance, fuel_efficiency, fuel_price)` that computes total cost. Base fee is $5. Add a 10% surcharge to the total cost (base fee + fuel cost) if the distance > 100.

Objective: Use conditional logic inside functions based on real-world constraints.

Input Format	Three floats: distance, fuel_efficiency (km/l), fuel_price ($/l).
Output Format	Total cost rounded to 2 decimal places.
Constraints	`All inputs > 0`

Example Input	`120, 15, 1.5`
Example Output	`18.70`
Explanation	Fuel needed = 120/15 = 8l. Fuel cost = 8*1.5 = $12. Base fee = $5. Total = $17. Surcharge (10%) = $1.7. Final = 18.70.

Edge Cases to Consider: Distance exactly 100 (no surcharge).

Additional Test Cases:

```
1) 50, 10, 2 -> 15.00
```

Hint: Calculate base + fuel cost first, then check if distance > 100 for surcharge.

> **Intuition & Analogy**
>
> *Analogy: Think of it like calculating a restaurant bill. You have the food cost (fuel), the table charge (base fee), and an automatic gratuity if your party is large enough (distance > 100).*

Sum of List Elements

Easy

Problem Statement: Write a program to compute the sum of all elements in a given list of integers without using the built-in sum() function.

Objective: Understand list traversal and accumulator variables.

Input Format	A list of N integers.
Output Format	An integer representing the sum.
Constraints	`0 <= N <= 10^5, -10^4 <= arr[i] <= 10^4`

Example Input	`[1, 2, 3, 4, 5]`
Example Output	`15`
Explanation	1+2+3+4+5 = 15

Edge Cases to Consider: Empty list (should return 0).

Additional Test Cases:

```
1) [] -> 0
2) [-1, 1] -> 0
```

Hint: Initialize an accumulator variable to 0 and add each element iteratively.

> **Intuition & Analogy**
>
> *Analogy: Imagine a piggy bank (the accumulator). As you walk down a line of people (the list), each person hands you coins (the elements), and you drop them into your piggy bank to get the total.*

Remove Duplicate in a List

Medium

Problem Statement: Write a program to remove duplicate elements from a list while maintaining the original order of first occurrences.

Objective: Practice list manipulation and tracking seen elements.

Input Format	A list of N integers.
Output Format	A list of integers with duplicates removed.
Constraints	`1 <= N <= 10^4`

Example Input	`[4, 3, 2, 4, 1, 3]`
Example Output	`[4, 3, 2, 1]`
Explanation	4 and 3 appear twice, only their first occurrences are kept.

Edge Cases to Consider: List with all identical elements.

Additional Test Cases:

```
1) [1, 1, 1] -> [1]
```

Hint: Use an auxiliary data structure like a Set to keep track of seen elements.

> **Intuition & Analogy**
>
> *Analogy: Think of acting as a bouncer at a club. You hold a clipboard (a 'seen' set). As people (numbers) arrive, you check your clipboard. If they aren't on it, you let them in and write their name down. If they are already on it, you turn them away.*

Real-World: Inventory Management

Medium

Problem Statement: You are given two lists: `items` (strings) and `quantities` (ints). Merge them into a list of tuples representing inventory. Ignore items with quantity <= 0.

Objective: Parallel list iteration and data filtering.

Input Format	Two lists of length N.
Output Format	A list of tuples: [(item, quantity), ...]
Constraints	`1 <= N <= 10^3`

Example Input	`['Apple', 'Banana', 'Milk'], [10, 0, 5]`
Example Output	`[('Apple', 10), ('Milk', 5)]`
Explanation	Banana has 0 quantity, so it is filtered out.

Edge Cases to Consider: Both lists are empty.

Additional Test Cases:

```
1) ['A'], [-1] -> []
```

Hint: Use a loop with zip() or range(len(items)).

> **Intuition & Analogy**
>
> *Analogy: Imagine two conveyor belts side-by-side. One has product labels, the other has weight scales. You pick up the label and the weight simultaneously. If the weight is 0, you throw both in the trash; otherwise, you stick the label on the box.*

Check for Prime Number

Medium

Problem Statement: Write a program to determine if a given integer N is prime.

Objective: Implement mathematical logic and optimization (checking up to sqrt(N)).

Input Format	An integer N.
Output Format	Boolean: True if prime, False otherwise.
Constraints	`1 <= N <= 10^9`

Example Input	`29`
Example Output	`True`
Explanation	29 has no divisors other than 1 and 29.

Edge Cases to Consider: N=1 (not prime), N=2 (even prime).

Additional Test Cases:

```
1) N=1 -> False
2) N=15 -> False
```

Hint: You only need to check for factors up to the square root of N.

> **Intuition & Analogy**
>
> *Analogy: If you are looking for factors of 36, they come in pairs (2x18, 3x12, 4x9, 6x6). Past 6 (the square root), the pairs just reverse (9x4). So, once you check up to the square root, you've seen half of every possible pair!*

GCD of Two Numbers

Medium

Problem Statement: Write a program to find the Greatest Common Divisor (GCD) of A and B using the Euclidean Algorithm.

Objective: Understand recursive math formulas and optimization.

Input Format	Two integers A and B.
Output Format	An integer representing the GCD.
Constraints	`0 <= A, B <= 10^9`

Example Input	`48, 18`
Example Output	`6`
Explanation	Divisors of 48: 1,2,3,4,6,8,12,16,24,48. Divisors of 18: 1,2,3,6,9,18. Common: 1,2,3,6. Max: 6.

Edge Cases to Consider: One of the numbers is 0.

Additional Test Cases:

```
1) 0, 5 -> 5
```

Hint: Repeatedly apply B = A % B until B is 0.

> **Intuition & Analogy**
>
> *Analogy: Imagine trying to tile a 48x18 floor with the largest possible square tiles. You place 18x18 tiles until you can't, leaving a 12x18 gap. Then use 12x12 tiles, leaving a 12x6 gap. Finally, 6x6 tiles fit perfectly. The 6 is your GCD.*

Check Palindrome

Easy

Problem Statement: Write a program to check if a string is a palindrome (reads the same forwards and backwards), ignoring case and spaces.

Objective: String slicing, casing, and two-pointer basics.

Input Format	A string S.
Output Format	Boolean: True or False.
Constraints	`1 <= len(S) <= 10^4`

Example Input	`'Race car'`
Example Output	`True`
Explanation	'Race car' without spaces and lowercased is 'racecar', which is a palindrome.

Edge Cases to Consider: String with only punctuation or single character.

Additional Test Cases:

```
1) 'a' -> True
2) 'Hello' -> False
```

Hint: Clean the string first (remove spaces, make lowercase), then check if S == S[::-1].

> **Intuition & Analogy**
>
> *Analogy: A palindrome is like a mirror reflection. If you put a mirror exactly in the middle of the word, the left side should perfectly reflect the right side.*

Real-World: Log File Parser

Hard

Problem Statement: Given a log string format '[ERROR] User login failed at 10:45', extract all error messages. Write a function that takes a list of log strings and returns a list of only the messages tagged with '[ERROR]'.

Objective: String matching and parsing practical data.

Input Format	List of log strings.
Output Format	List of extracted error messages.
Constraints	`Logs follow '[LEVEL] Message' format.`

Example Input	`['[INFO] App started', '[ERROR] Disk full']`
Example Output	`['Disk full']`
Explanation	Only the second string contains '[ERROR]'. The prefix is stripped.

Edge Cases to Consider: Empty logs, no errors present.

Additional Test Cases:

```
1) ['[WARN] Low memory'] -> []
```

Hint: Use string methods like .startswith('[ERROR] ') to ensure the space is included when slicing.

> **Intuition & Analogy**
>
> *Analogy: Think of filtering mail. You scan the envelope (the prefix tag). If it says 'JUNK' (INFO), you throw it out. If it says 'URGENT' (ERROR), you open it and read the letter (extract the message).*

SECTION 6 — MINI PROJECTS

Project 1: Student Management System

Hard

Problem Statement: Build a CLI application integrating Functions, Lists, Dictionaries, and Loops. The system should allow a user to: 1) Add student records (Name, ID, Marks). 2) View all students. 3) Search student by ID. 4) Calculate class average.

Objective: Combine multiple data structures and implement a continuous menu loop.

> **Intuition & Analogy**
>
> *Think of this like building a digital filing cabinet. The outer loop is the receptionist asking "How can I help you?". The dictionaries are the individual manila folders holding student details, and the list is the drawer holding all the folders.*

Student Workspace (Algorithm / Rough Notes)

SECTION 7 — DEBUG THE CODE

Debugging 1: Infinite Loop

Medium

Problem Statement: The following code is supposed to sum digits of a number, but it hangs forever. Identify the missing state update and review the corrected solution below.

Buggy Code:

```
def sum_digits(n):
    total = 0
    while n > 0:
        total += n % 10
        # Missing state update!
    return total
```

Corrected Solution:

```
def sum_digits(n):
    total = 0
    while n > 0:
        total += n % 10
        n = n // 10  # BUG FIXED: Added state update to decrement n
    return total
```

Student Workspace (Algorithm / Rough Notes)